CS5446: AI Planning and Decision Making

AY25/26 Semester 2

MiniGrid

Group 35

Name	Student Number
CHAN KENG JWEE, BENEDICT	A0249460B
CHANG XIAOYIN	A0318376W
GU HAIWEI	A0245155H

1. INTRODUCTION AND ENVIRONMENT ANALYSIS

MiniGrid is a grid world benchmark for sequential decision making under partial observability, where the agent only receives a local egocentric 7 x 7 view. In this project, the six environments test different capabilities, including navigation, exploration, object interaction, safety-aware control, adaptation to dynamic changes, and memory.

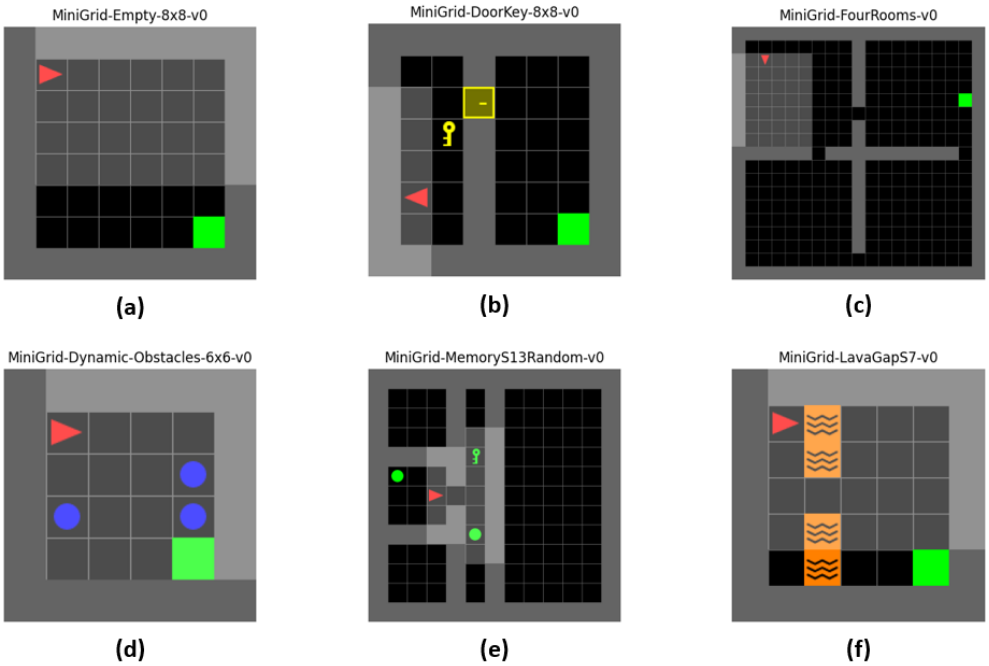

Fig. 1: MiniGrid visual examples of (a) Empty (b) Door Key (c) Four Rooms (d) Dynamic Obstacles (e) Memory (f) Lava Gap

1.1 EMPTY MINIGRID

Empty is an open room with a goal square and no hazards or interaction objects. Its main value is as a baseline where failure here usually indicates weak turning, orientation, or movement control rather than high-level reasoning problems. Although simple, it still requires minimal exploration under partial observability, so it tests whether the policy has learned a stable navigation prior.

1.2 DOORKEY MINIGRID

Door Key requires the agent to find a key, pick it up, open a door, and only then move to the goal. The task therefore introduces explicit subgoal ordering where the agent must complete prerequisite actions before goal-seeking becomes meaningful. DoorKey` is important because it tests hierarchical decision-making and correct use of interaction actions, not merely locomotion.

1.3 FOUR ROOMS MINIGRID

Four Rooms divides the map into four, connected by narrow doorways. The difficulty is topological rather than geometric since an action that appears locally reasonable may still be globally poor if it moves the agent away from the correct doorway. This therefore tests whether the policy can tolerate temporary detours and reason about connectivity instead of following a purely greedy direction-to-goal heuristic.

1.4 DYNAMIC OBSTACLES MINIGRID

Dynamic Obstacles extends navigation with moving obstacles, so safety depends on both spatial position and timing. The key difficulty is reactive adaptation where an action that is safe now may be poor one step later. It therefore evaluates closed-loop control under uncertainty, including whether the agent can wait, sidestep, or exploit short openings instead of committing to a fixed path.

1.5 MEMORY MINIGRID

Memory gives the agent relevant information early in the episode and requires it to use that information later, after the cue is no longer visible. Its main challenge is state aliasing as two late observations may look identical while requiring different actions depending on earlier experience. Consequently, this tests whether the policy can maintain useful internal state rather than acting only on the current frame.

1.6 LAVAGAP MINIGRID

Lava Gap places lava between the agent and the goal, leaving only a narrow safe route. It primarily tests risk-sensitive planning, since locally direct motion toward the goal may be unsafe. Strong performance therefore requires the agent to trade off immediate progress against long-term survivability rather than simply follow goal attraction.

2. TECHNIQUES

2.1 CLASSICAL APPROACH

A classical Breadth First Search (BFS) agent was explored early in development. It incrementally mapped observed cells, searched over position and orientation states, prioritised key-door-goal sequencing for DoorKey, and otherwise planned toward the goal or nearest unseen cell. This approach was feasible in MiniGrid's small discrete state space, but it was not retained because it handled partial observability, dynamic changes, and history dependence less effectively than the recurrent learned policy.

2.2 LEARNED POLICY ARCHITECTURE

The final solution is a task-conditioned recurrent actor-critic, deployed as a compact two expert policy system rather than as a single monolithic checkpoint. The submission exposes a single controller that internally routes between a shared base expert and a DoorKey specialist using only public observations. This split matches the structure of the benchmark where most tasks can be expressed with a compact navigation action set, whereas DoorKey additionally requires key pickup and door toggling and therefore imposes a qualitatively different control problem with explicit subgoal ordering. At inference time the controller is purely learned, with no planner or symbolic fallback.

At the model level, the policy operates on MiniGrid's symbolic observation tensor rather than raw pixels. The object, colour, and state channels of the local 7 x 7 view are embedded separately, concatenated, and processed by a two-layer convolutional encoder. The resulting spatial representation is fused with direction and task embeddings before entering a hidden layer and recurrent backbone based on Gated Recurrent Units (GRUs). This gives the network both spatial structure and task awareness while the recurrent state provides memory for partially observed tasks such as Memory and for delayed decision structure in environments such as FourRooms. In the final DoorKey consolidation stage, these task-specific components are enabled for DoorKey, allowing limited specialisation while preserving a shared navigation representation for the other environments.

2.3 LEARNED TRAINING PROCEDURE

Training follows a staged imitation-to-reinforcement pipeline. Demonstration data are generated by environment-specific oracle teachers such as shortest path navigation for simpler maps, a causal teacher for Memory, a one-step lookahead teacher for Dynamic Obstacles, and a three-phase teacher for DoorKey that acquires the key, opens the door, and only then proceeds to the goal. These demonstrations are repackaged as padded episode sequences and used for recurrent behaviour cloning with cross-entropy loss, before Proximal Policy Optimisation (PPO) finetunes the policy using generalised advantage estimation, a clipped surrogate objective, value regression, entropy regularization, and gradient clipping. In the final retained stages, PPO used vectorised rollouts of 15-20 environments, horizons of 256-512 steps, 4 PPO epochs per update, learning rate 1e-4, and checkpoint selection by weighted per-task success rather than average reward alone.

The final training proceeded in two stages. First, a five-task PPO stage strengthened the shared recurrent navigation policy using a FourRooms-heavy task mix. Second, a DoorKey-focused distillation stage resumed from that five-task base, expanded the action space, applied additional behaviour cloning, and distilled the earlier policy on the original five tasks while training DoorKey-specific capacity. This produced a deliberately structured multi-task pipeline rather than a generic one-pass PPO run.

3. RESULTS

3.1 QUANTITATIVE EVIDENCE

The final agent was evaluated locally for 1000 episodes on each of the six environments. Table 1 below reports success rate, average reward, reward standard deviation, and average inference time per episode. Because MiniGrid reward depends on both success and efficiency, success rate is treated as the primary metric and reward as a secondary indicator of solution quality.

Environment	Episodes	Success Rate	Avg Reward	Std Reward	Avg Time
Empty	1000	100.00	0.961328	0	0.014005
Door Key	1000	98.90	0.963510	0.101854	0.042428
Four Rooms	1000	88.20	0.645264	0.302457	0.055100
Dynamic Obstacles	1000	99.90	0.871012	0.084703	0.024476
Lava Gap	1000	100.00	0.952043	0.003553	0.016114
Memory	1000	100.00	0.987027	0.003963	0.021794

Table 1: Performance for all 6 environments across 1000 episodes

The measured results support three main conclusions. First, the final agent achieved perfect success in this 1000-episode evaluation on Empty, LavaGap, and Memory, indicating strong performance on basic navigation, safety-aware route selection, and explicit memory dependence. Second, performance remained near-perfect on Dynamic Obstacles (99.9%) and DoorKey (98.9%), showing that the policy also performed well in settings involving moving hazards and sequential object interaction. Third, FourRooms remained the principal bottleneck at 88.2% success, with substantially lower mean reward and higher reward variance than the other environments. This suggests that global connectivity and long-horizon exploration were more difficult for the final policy than reactive control or short-horizon goal pursuit.

3.2 COMPARATIVE ANALYSIS

The distribution of evaluation times is also informative. Mean episode time ranges from approximately 0.014 s in Empty to 0.055 s in FourRooms, which indicates that the final agent remains lightweight at inference time. DoorKey is slower than the simpler navigation tasks, which is expected because it requires longer sequential interaction and uses a specialised branch with an expanded action set.

The final training configuration placed additional emphasis on FourRooms and DoorKey through heavier sampling, extra behaviour cloning, distillation, and task-specific capacity. This is consistent with the observed outcome where DoorKey reached 98.9% success while FourRooms remained the main residual weakness.

4. DISCUSSIONS

4.1 STRENGTHS

The strongest aspect of the final system is that it did not force the benchmark into a purely uniform policy and instead used structure where the environments required it. Symbolic channel embeddings fit MiniGrid's observation format, recurrence addresses partial observability, and task conditioning allows substantial sharing across environments. In the reported evaluation, this design was particularly effective on Memory, LavaGap, and Dynamic Obstacles, and remained strong on DoorKey. More broadly, the project suggests that on a small partially observed benchmark, the right inductive bias can matter more than increasing model size.

4.2 LIMITATIONS

At the same time, the final evaluation makes clear that complete unification remained difficult. The most obvious sign is the persistent FourRooms gap. Although the agent solves the other environments at high success rates, FourRooms remains noticeably weaker and shows the highest reward variance. The deployed controller is also not one fully shared six-task policy, but a single submission controller that routes internally between a five-task base expert and a DoorKey-specialised expert. This is a pragmatic engineering choice but it also implies that the strongest results depended on manual task balancing and selective specialisation rather than on a single uniformly strong policy.

The clearest remaining weakness is deployment-time task inference. The controller identifies tasks from mission text and, in the ambiguous Empty versus Dynamic Obstacles case, from whether obstacle IDs are currently visible in the local observation. A stronger next step would

be to learn an episode-level gating or belief state that infers task identity jointly with action selection, instead of relying on hand-written rules derived from surface observations.

4.3 FUTURE IMPROVEMENTS

The next technical step is to reduce the amount of hand-crafted structure in the final controller. In particular, the current system still relies on explicit task routing and a separate DoorKey specialist, which solves the benchmark effectively but stops short of a fully unified policy. A stronger follow-up would replace hand-written task inference with a learned latent-task or mixture-of-experts (MOE) gating mechanism, while also testing whether the same specialisation can be recovered inside a single recurrent policy with shared parameters. A second promising direction is to strengthen exploration and memory directly, for example through auxiliary objectives or belief-state modelling, so that environments such as FourRooms can be improved without relying as heavily on manual task balancing or behavior-cloning support.

References

[1] M. Chevalier-Boisvert, B. Dai, M. Towers, R. de Lazcano, L. Willems, S. Lahlou, S. Pal, P. S. Castro, and J. Terry, "Minigrid & Miniworld: Modular & Customizable Reinforcement Learning Environments for Goal-Oriented Tasks," arXiv, 2023. Available: https://arxiv.org/abs/2306.13831

[2] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal Policy Optimization Algorithms," arXiv, 2017. Available: https://arxiv.org/abs/1707.06347

[3] J. Schulman, P. Moritz, S. Levine, M. Jordan, and P. Abbeel, "High-Dimensional Continuous Control Using Generalized Advantage Estimation," arXiv, 2015. Available: https://arxiv.org/abs/1506.02438

[4] K. Cho, B. van Merrienboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio, "Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation," arXiv, 2014. Available: https://arxiv.org/abs/1406.1078

[5] S. Ross, G. Gordon, and D. Bagnell, "A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning," in Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics, 2011. Available: https://proceedings.mlr.press/v15/ross11a.html